

Kapitel 7. Funktioner

Innehållsförteckning

[Grundbegrepp](#)[Anropa funktioner](#)[Parametertyper](#)[Funktionen `main\(\)`](#)[Parametrar till `main\(\)`](#)[Omfattning \(scoping\)](#)[Rekursion](#)[Funktioner som parametrar](#)

Detta kapitel behandlar funktioner i C++. Funktioner är den grundläggande formen av modularisering i C++.

Grundbegrepp

I de flesta programmeringsspråk finns det olika sätt att ordna kod i olika separata block eller moduler som associeras med ett namn. Dessa brukar kallas *funktioner* eller *procedurer*, beroende på språket. Även i C++ finns det funktioner. De är den grundläggande byggstenen för modulariserade program och ger programmeraren de första möjligheterna att göra större programhelheter. En funktion i C++ kategoriseras av att den har ett *namn*, ett *returnvärde* (se [avsnittet Returvärden](#)) och ett antal *parametrar* (se [avsnittet Parametrar till funktioner](#)). Den allmänna formen för en funktion ser ungefär ut så här:

```
returntyp funktionsnamn (parametertyp1 parameter1, ... ) {  
    satser  
}
```

Funktioner kan namnges precis på samma sätt som variabler. De kan innehålla alfabetiska tecken, siffror och `_`, och följer samma regler som variabler.

När det gäller funktioner är `{` och `}` obligatoriska för att avgränsa det som hör till funktionen. Likaså är en tom parentes `()` obligatorisk även om funktionen inte tar några parametrar.

Parametrar till funktioner

För att funktioner skall kunna göra någon nytta behöver de för det mesta ett antal parametrar som är det data de jobbar med. Följande enkla funktion tar inga parametrar överhuvudtaget, och skriver endast ut en enkel text:

```
void Print1 () {  
    cout << "Hej från Print1!" << endl;  
}
```

Ovanstående funktion tar inga parametrar, och dess parameterlista är således en tom parentes `()`. Vi kan ändra funktionen så att den tar en parameter av typen `int` som även skrivs ut:

```
void Print2 (int Tal) {  
    cout << "Hej från Print2, talet är:" << Tal << endl;  
}
```

Nu börjar funktionen redan göra någon nytta. Parametern `Tal` kan användas som en helt normal variabel av koden i `Print2`, d.v.s. den kan bl.a. tilldelas och skrivas ut. Låt oss modifiera funktionen så att den tar en valfri text som parameter, samt ett tal som anger hur många gånger texten skall skrivas ut:

```
void Print3 (string Text, int Tal) {  
    int Index;  
    for ( Index = 0; Index < Tal; Index++ ) {  
        cout << Text << endl;  
    }  
}
```

Parametrar separeras alltså med ett kommatecken (,). För funktionens del är det ingen skillnad i vilken ordning parametrarna finns. Exemplet ovan introducerar även en *lokal variabel*, men vi kommer till det senare. `Index` används till att iterera `Tal` antal gånger och skriva ut `Text`. Låt oss nu göra en funktion som beräknar en rektangels area:

```
void RektangelAreal (float Kant1, float Kant2) {  
    // beräkna radien och skriv ut  
    float Area = Kant1 * Kant2;  
    cout << "Areal är: " << Area << endl;  
}
```

Detta börjar redan närma sig en funktion som vi kan se en viss nytta med. En allvarlig brist är dock att vi inte hittills kan förmedla resultatet tillbaka till anroparen. Det är inget problem, läs vidare.

Returvärden

Funktioner är ganska värdelösa om de inte kan förmedla returvärden tillbaka till den som anropade funktionen. Redan namnet *funktion* implicerar ju att det är frågan om någonting som utför en operation och returnerar någonting. I exemplen tidigare användes inget returvärde, utan returtypen var deklarerad som `void`. Denna datatyp betyder att ingenting returneras, att returvärdet inte helt enkelt finns. Vi skall nu göra om vår areaberäknande funktion så att den returnerar ett värde:

```
float RektangelArea2 (float Kant1, float Kant2) {  
    // beräkna radien och skriv ut  
    float Area = Kant1 * Kant2;  
  
    // returnera arean  
    return Area;  
}
```

Nu är returtypen definierad till `float`. Nytt är även att sist i funktionen utförs satsen `return`. Denna används för att förmedla ett returvärde, som placeras efter `return`. När denna sats exekveras avbryts funktionen omedelbart och exekveringen fortsätter där funktionen anropades. I funktioner som inte returnerar någonting alls kan `return` även användas för att avbryta exekveringen av funktionen, men då ges inget värde till `return`. Vi kan modifiera vår `Print3` och göra den lite mera robust:

```
void Print4 (string Text, int Tal) {  
    int Index;  
    // verifiera att Tal är > 0
```

```
if ( Tal <= 0 ) {  
    cout << "Aj, aj, kan inte skriva ut något..." << endl;  
    return;  
}  
else {  
    for ( Index = 0; Index < Tal; Index++ ) {  
        cout << Text << endl;  
    }  
}  
}
```

Nu använder vi oss av en tom `return` ifall användaren gett in ett `Tal` som är `<= 0`. I så fall kan ju inget skrivas ut. Funktionen `Print4` har returdatatypen `void`, så det är ogiltigt att försöka returnera någonting. På samma vis bör en funktion som enligt sin definition skall returnera ett returvärde av en viss typ även göra det.

Man kan returnera olika typer av data, t.ex. variabler, konstanter o.s.v. Kravet är att den "mottagande" variabeln är av samma typ som den typ som funktionen returnerar, eller av en typ till vilken det går att konvertera automatiskt.

[Förutgående](#)[Hem](#)[Nästa](#)

? :-operatörn

Anropa funktioner